

Advanced Loop Lesson – Mastering Java

Loops with Star Patterns

This is a **hands-on, advanced loop lesson** in Java. We'll master loops via progressively complex `star( * )` examples.

Design rules used throughout:

- Index from **1** up to **`<= max`**.
- Use **descriptive variable names**: `row`, `rowMax`, `column`, `columnMax`, `widthMax`, `heightMax`, `depthMax`.
- For each printed line, **build the full string first (StringBuilder)**, then print once.
- Add **clear comments** on non-obvious conditions and formulas.

1) Example: Print 100 stars sequentially (single line)

Goal

Print exactly 100 `*` characters in a single line.

```
public static void printHundredStarsSingleLine() {
    int countMax = 100;
    StringBuilder builder = new StringBuilder(countMax);
    for (int column = 1; column <= countMax; column++) {
        builder.append('*');
    }
    System.out.println(builder.toString());
}

// Demo
public static void demoExample1() {
    System.out.println("Example 1: 100 stars in one line");
    printHundredStarsSingleLine();
    System.out.println();
}
```

Key idea: When you know the **exact count**, use a loop or direct repetition (here: loop + StringBuilder).

2) Example: Print 100 stars in a 10 × 10 grid

Goal

Arrange 100 stars as a 10-row by 10-column grid.

```
public static void printStarsGrid10x10() {
    int rowMax = 10;
    int columnMax = 10;
    for (int row = 1; row <= rowMax; row++) {
        StringBuilder builder = new StringBuilder(columnMax);
        // Build one line with exactly 10 '*' characters
        for (int column = 1; column <= columnMax; column++) {
            builder.append('*');
        }
        System.out.println(builder.toString());
    }
}

// Demo
public static void demoExample2() {
    System.out.println("Example 2: 10x10 grid of stars");
    printStarsGrid10x10();
    System.out.println();
}
```

Loop roles:

- **Outer loop** drives **rows** (lines).
 - **Inner loop** builds **columns** (characters per line).
-

3) Example: From a list of integers, print those many stars – one star per new line

Goal

Given a list/array of non-negative integers, print **that many lines** each containing a single * for each number.

To avoid ambiguity, we include a **popular interview variant** as well.

Variant A – One star per line (as specified)

```
public static void printOneStarPerLineFromCounts(int[] counts) {
    if (counts == null) {
        System.out.println("(no data)");
        return;
    }
    for (int row = 1; row <= counts.length; row++) {
        int linesToPrint = Math.max(0, counts[row - 1]); // clamp negative
        for (int column = 1; column <= linesToPrint; column++) {
            System.out.println('*');
        }
    }
}

// Demo
public static void demoExample3A() {
    System.out.println("Example 3A: one star per line per count");
    printOneStarPerLineFromCounts(new int[]{3, 1, 4});
    System.out.println();
}
```

Variant B – All stars for a number on the same line (popular interview variant)

```
public static void printStarsPerCountOnOneLine(int[] counts) {
    if (counts == null) {
        System.out.println("(no data)");
        return;
    }
    for (int row = 1; row <= counts.length; row++) {
        int columnMax = Math.max(0, counts[row - 1]);
        StringBuilder builder = new StringBuilder(columnMax);
        for (int column = 1; column <= columnMax; column++) {
            builder.append('*');
        }
    }
}
```

```

        System.out.println(builder.toString());
    }
}

// Demo
public static void demoExample3B() {
    System.out.println("Example 3B: stars per count on one line");
    printStarsPerCountOnOneLine(new int[]{3, 1, 4, 0, 2});
    System.out.println();
}

```

4) Example: Print stars alternatively (skip one, print one)

Goal

Given a length (e.g., 20), print positions 1, 3, 5, ... as ***** and positions 2, 4, 6, ... as spaces.

```

public static void printAlternateStars(int lengthMax) {
    if (lengthMax < 0) lengthMax = 0;

    StringBuilder builder = new StringBuilder(lengthMax);
    for (int column = 1; column <= lengthMax; column++) {
        boolean isOddPosition = (column % 2 == 1);
        builder.append(isOddPosition ? '*' : ' ');
    }
    System.out.println(builder.toString());
}

// Demo
public static void demoExample4() {
    System.out.println("Example 4: alternate stars (length 20)");
    printAlternateStars(20);
    System.out.println();
}

```

Extension: Apply the same idea to a grid (e.g., a checkerboard).

```

public static void printAlternateStarsGrid(int heightMax, int widthMax) {
    for (int row = 1; row <= heightMax; row++) {

```

```

        StringBuilder builder = new StringBuilder(widthMax);
        for (int column = 1; column <= widthMax; column++) {
            boolean isOddSum = ((row + column) % 2 == 1);
            builder.append(isOddSum ? '*' : ' ');
        }
        System.out.println(builder.toString());
    }
}

// Demo
public static void demoExample4Grid() {
    System.out.println("Example 4 (grid): alternate stars checkerboard 8x16");
    printAlternateStarsGrid(8, 16);
    System.out.println();
}

```

5) Example: Print stars only at the edges (hollow rectangle)

Goal

Given `heightMax` × `widthMax` , print stars on the **border** only.

```

public static void printHollowRectangle(int heightMax, int widthMax) {
    for (int row = 1; row <= heightMax; row++) {
        StringBuilder builder = new StringBuilder(widthMax);
        for (int column = 1; column <= widthMax; column++) {
            boolean isBorder = (row == 1 || row == heightMax || column == 1 || column == widthMax);
            builder.append(isBorder ? '*' : ' ');
        }
        System.out.println(builder.toString());
    }
}

// Demo
public static void demoExample5() {
    System.out.println("Example 5: hollow rectangle 6x12");
    printHollowRectangle(6, 12);
    System.out.println();
}

```

6) Example: Print diagonals and edges

Goal

On an $N \times N$ grid, print both diagonals **and** the border.

- Main diagonal: `row == column`
- Anti-diagonal: `row + column == N + 1`

```
public static void printDiagonalsAndEdges(int n) {
    int rowMax = n;
    int columnMax = n;

    for (int row = 1; row <= rowMax; row++) {
        StringBuilder builder = new StringBuilder(columnMax);
        for (int column = 1; column <= columnMax; column++) {
            boolean onEdge = (row == 1 || row == rowMax || column == 1 ||
                               column == columnMax);
            boolean onMainDiag = (row == column);
            boolean onAntiDiag = (row + column == n + 1);

            if (onEdge || onMainDiag || onAntiDiag) {
                builder.append('*');
            } else {
                builder.append(' ');
            }
        }
        System.out.println(builder.toString());
    }
}

// Demo
public static void demoExample6() {
    System.out.println("Example 6: diagonals and edges (n=9)");
    printDiagonalsAndEdges(9);
    System.out.println();
}
```

7) Triangle Patterns

The building blocks are **spaces** and **stars** per row. Always derive counts from `row` and `rowMax`.

7.1 Left-aligned triangle

Row `r` has exactly `r` stars.

```
public static void printLeftAlignedTriangle(int rowMax) {
    for (int row = 1; row <= rowMax; row++) {
        int starCount = row; // grows with the row
        StringBuilder builder = new StringBuilder(starCount);
        for (int column = 1; column <= starCount; column++) {
            builder.append('*');
        }
        System.out.println(builder.toString());
    }
}

// Demo
public static void demoExample7_1() {
    System.out.println("Example 7.1: left-aligned triangle (rowMax=5)");
    printLeftAlignedTriangle(5);
    System.out.println();
}
```

7.2 Right-aligned triangle

Spaces first, then stars.

- `spaceCount = rowMax - row`
- `starCount = row`

```
public static void printRightAlignedTriangle(int rowMax) {
    for (int row = 1; row <= rowMax; row++) {
        int spaceCount = rowMax - row;
        int starCount = row;

        StringBuilder builder = new StringBuilder(spaceCount + starCount)
        for (int i = 1; i <= spaceCount; i++) builder.append(' ');
        for (int i = 1; i <= starCount; i++) builder.append('*');

        System.out.println(builder.toString());
    }
}
```

```

}

// Demo
public static void demoExample7_2() {
    System.out.println("Example 7.2: right-aligned triangle (rowMax=5)");
    printRightAlignedTriangle(5);
    System.out.println();
}

```

7.3 Centered pyramid

- `leftSpaces = rowMax - row`
- `stars = 2 * row - 1`

```

public static void printCenteredPyramid(int rowMax) {
    for (int row = 1; row <= rowMax; row++) {
        int leftSpaces = rowMax - row;
        int starCount = 2 * row - 1;

        StringBuilder builder = new StringBuilder(leftSpaces + starCount)
        for (int i = 1; i <= leftSpaces; i++) builder.append(' ');
        for (int i = 1; i <= starCount; i++) builder.append('*');

        System.out.println(builder.toString());
    }
}

// Demo
public static void demoExample7_3() {
    System.out.println("Example 7.3: centered pyramid (rowMax=5)");
    printCenteredPyramid(5);
    System.out.println();
}

```

7.4 Inverted centered pyramid

- `leftSpaces = row - 1`
- `stars = 2 * (rowMax - row) + 1`

```

public static void printInvertedCenteredPyramid(int rowMax) {
    for (int row = 1; row <= rowMax; row++) {

```



```

        int leftSpaces = row - 1;
        int starCount = 2 * (rowMax - row) + 1;

        StringBuilder builder = new StringBuilder(leftSpaces + starCount)
        for (int i = 1; i <= leftSpaces; i++) builder.append(' ');
        for (int i = 1; i <= starCount; i++) builder.append('*');

        System.out.println(builder.toString());
    }
}

// Demo
public static void demoExample7_4() {
    System.out.println("Example 7.4: inverted centered pyramid (rowMax=5)");
    printInvertedCenteredPyramid(5);
    System.out.println();
}

```

8) How to Derive Loop Logic (Recipe)

1. **Sketch small cases** (`rowMax = 3` or `4`).
 2. Decide **for each row**: how many **spaces** and how many **stars**?
 3. Turn those counts into **two loops** (spaces first, then stars).
 4. Keep **outer loop = rows**, **inner loops = counts**.
 5. Maintain the **1.. <= max** pattern to avoid off-by-one errors.
 6. **Build then print** each line to avoid partial outputs and simplify debugging.
-

9) Debugging Off-by-One & Alignment

- If the shape is **shifted**, re-check **space counts**.
 - If the shape is **too wide or narrow**, re-check **star formulas** (e.g., `2 * row - 1`).
 - Always test **tiny heights** (`rowMax = 1, 2, 3, 4`) to spot mistakes early.
-

10) Bonus Variations To Practice

- **Hollow pyramid** (stars only at left edge, right edge, and base).
- **Diamond** (pyramid up + inverted pyramid down; avoid duplicating the middle row).
- **Checkerboard** with tunable cell size.

- **Numbers instead of stars** (e.g., print `row` or `column` values).
-

11) Mini Runner (Optional)

```
public class StarPatternJavaDemo {  
  
    public static void main(String[] args) {  
        demoExample1();  
        demoExample2();  
  
        demoExample3A();  
        demoExample3B();  
  
        demoExample4();  
        demoExample4Grid();  
  
        demoExample5();  
        demoExample6();  
  
        demoExample7_1();  
        demoExample7_2();  
        demoExample7_3();  
        demoExample7_4();  
    }  
  
    // Paste all the methods from the sections above here.  
}
```

Final Tip

Think **grid-first**: *What should appear at (row, column)?* Once counts are right, loops are just implementation details.